

MILESTONE 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities

1. Legal Document Generation

Functionality: Generate legal documents (e.g., contracts, agreements) based on user inputs (document type, parties, terms, and dates).

```
# routes.py
@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}

# gemini_generator.py
def generate_document(self, document_type, parties, terms, dates):
    prompt = (
        f"Generate a comprehensive legal document titled '{document_type}'\n"
        f"Involved parties: {parties}\n"
        f"Effective Date: {dates}\n"
        f"Terms and conditions: {terms}\n"
        f"Ensure formal legal structure with multiple sections and legal clauses."
    )
    response = self.model.generate_content(prompt)
    return response.text
```

2. Document Output Formatting

Functionality: Convert raw AI-generated text into structured .docx, .pdf, .txt files with legal formatting, logos, tables, and footers.

- format_docx(text, doc_type)
- format_pdf(text, doc_type)

- `format_html_preview(text)`

These use:

- **docx** for Microsoft Word formatting (includes logo, Times New Roman font, terms as table, footer).
- **fpdf** for clean, branded PDF documents with headers/footers.
- **HTML Preview** with Streamlit styling.

3. User Interaction & Recommendations

Functionality: Accept user preferences through the Streamlit interface and reflect them in the AI-generated legal content.

```
# app.py - Streamlit UI
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions")
dates = st.text_input("Effective Date")

# Send to backend
response = requests.post("http://localhost:8000/generate", json={
    "document_type": document_type,
    "parties": parties,
    "terms": terms,
    "dates": dates
})
```

4. Editable Document & Download

Functionality: Provides editable text area, allows download in .txt, .docx, .pdf formats.

```
# app.py
if st.session_state.get("show_edit"):
    edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
    st.session_state.generated_text = edited_text

# Downloads
st.download_button("📄 Download as .TXT", data=generated_text, ...)
st.download_button("📄 Download as .DOCX", data=format_docx(...), ...)
st.download_button("📄 Download as .PDF", data=format_pdf(...), ...)
```

Activity 2.2: Implement the FastAPI Backend

1. Define Routes in FastAPI (Instead of Flask)

```
# main.py
app.include_router(router)

# routes.py
@router.post("/generate") # Route for generating legal document
```

2. Process User Input

Handled via Pydantic & Streamlit Forms:

```
# routes.py
class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str
```

```
# app.py
# Streamlit collects input → FastAPI handles it
response = requests.post("http://localhost:8000/generate", json={...})
```

3. Integrate Groq (Gemini) API Calls

```
# gemini_generator.py
genai.configure(api_key=os.getenv("GEMINI_API_KEY"))

self.model = genai.GenerativeModel(model_name)
response = self.model.generate_content(prompt)
```